Table 1-10. *Stream functions*

`stream_socket_client()`	Connect to an Internet or Unix domain socket
`stream_socket_server()`	Create an Internet or Unix server socket – allows clients to connect to your program
`stream_socket_recvfrom()`	Receive data from a server socket
`stream_socket_shutdown()`	Close the server socket
`stream_filter_register`	Register a user-defined class as a stream filter (like `php://filter`)
`stream_filter_prepend()` `stream_filter_append()`	Attach a registered filter to a stream
`stream_filter_remove()`	Remove a filter

There is one more form of I/O, which you will see a great deal of when working on PHP web applications: databases. At heart these are just another form of stream (usually a network connection or via a Unix domain socket file if the database server is running on the same machine) but have their own sets of specialized modules in PHP, which we'll be looking at next.

Databases

Data persistence (making sure data hangs around after your program is done) generally boils down to two forms: filesystem and database. It's perfectly fine to store data in files, and in fact, PHP can do that automatically for some types of data associated with HTTP requests. For example, user session data, as we'll see in Chapter 11, "Programming for the Internet." However, managing complex data becomes challenging quite quickly if everything is backed by files. Chapter 10, "Programming with Objects," will introduce concepts such as *domain modeling*, where software components are created to model the reality of the problem that the software is designed to solve. Usually those components represent real things such as people, places, and actions, and require that data records for those entities be maintained. Imagine spreading thousands of files across a complex filesystem layout and having to manage it every time the application serviced a request… It would be quite a headache.